

Колекції

Частина 1. Списки

Структура даних — це спосіб організації даних (разом з набором операцій над ними), при якому сукупність значень розглядається як одне ціле. Наприклад, рядок (тип `str`) — це структура даних, яка об'єднує послідовність символів. Python, крім рядків, підтримує інші вбудовані структури даних, зокрема: списки, кортежі, словники та множини. Усі вони використовуються для зберігання колекцій об'єктів, але мають свої унікальні особливості.

Список (тип `list`) — це впорядкована змінювана послідовність (колекція) об'єктів, які називаються елементами списку. Списки за логікою організації схожі на рядки, а як структура даних мають багато спільного з масивами в інших мовах програмування¹, але суттєво відмінні від останніх:

- немає необхідності наперед вказувати кількість елементів (довжину) списку;
 - тип елементів одного й того самого списку може бути різний;
 - списки — змінюваний тип, який підтримує операції додавання, видалення та зміни елементів.
-

Подання списків

Літералом списку в Python є послідовність об'єктів, записаних через кому у квадратних дужках, наприклад:

```
[3., 3.1, 3.14, 3.142, 3.1416]      # список наближень числа "pi"  
['зелений', 'жовтий', 'червоний'] # список кольорів світлофора  
[1, ['Yes', 'No'], 2.1, 'Maybe']  # список елементів різних типів  
[]                                  # порожній список
```

Список може бути записаний у кілька рядків:

```
rainbow = ['red', 'orange',  
           'yellow', 'green', 'blue',  
           'indigo', 'violet']
```

Якщо елементом списку є вираз, то інтерпретатор обчислюватиме його значення:

¹ Також не треба плутати списки Python зі зв'язними списками (linked lists)

```
x = 10
print([x-1, x**2-1, x**3-1]) # [9, 99, 999]
```

Списки можна об'єднувати, використовуючи оператор "+" (операція схожа на конкатенацію рядків):

```
pi = [3., 3.1, 3.14, 3.142, 3.1416]
pi = pi + [3.14159, 3.141593]
pi += [3.1415927]
print(pi)
# [3.0, 3.1, 3.14, 3.142, 3.1416, 3.14159, 3.141593, 3.1415927]
```

Оператор * надає можливість розмножувати список однаковими значеннями:

```
print([1, 'один']*3) # [1, 'один', 1, 'один', 1, 'один']
```

Список також може бути утворений з будь-якої послідовності елементів. Для цього потрібно передати цю послідовність функції list():

```
s = 'Це рядок.'
lst = list(s)
print(lst) # ['Ц', 'е', ' ', 'р', 'я', 'д', 'о', 'к', '.']
print(list(range(10))) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Індекси та зрізи

Кожен елемент списку має індекс — унікальне ціле число, яке однозначно вказує на порядковий номер елемента у списку. Індекси можна відраховувати з початку списку (тоді нумерація починається з нуля: 0, 1, 2, ...), а можна з кінця (тоді індекси від'ємні: -1, -2, -3 і т. д.):

```
spysok = [1, ['Yes', 'No'], 2.1, 'Maybe']
print(spysok[1]) # ['Yes', 'No']
print(spysok[0], spysok[2], spysok[-2]) # 1 2.1 2.1
```

Щоб отримати доступ до елементів вкладеного списку, індекси записуються послідовно:

```
print(spysok[1][0], spysok[1][1]) # Yes No
```

Оскільки список — змінюваний тип даних, то його елементи можна змінювати:

```
spysok[1] = 'Maybe'
print(spysok)                # [1, 'Maybe', 2.1, 'Maybe']
```

Якщо елемента з вказаним індексом не існує, буде ініційована помилка:

```
print(spysok[4])             # IndexError: list index out of range
```

Зрізи дають змогу отримати певну підмножину елементів списку. При цьому сам список не змінюється — результат зрізу записується у новий список. Синтаксис зрізу такий самий, як і для рядків:

```
rainbow = ['red', 'orange', 'yellow', 'green',
           'blue', 'indigo', 'violet']

print(colors[:])
# ['red', 'orange', 'yellow', 'green', 'blue', 'indigo', 'violet']
print(colors[:3])      # ['red', 'orange', 'yellow']
print(colors[2:4])     # ['yellow', 'green']
print(colors[1::2])    # ['orange', 'green', 'indigo']
print(colors[::-1])
# ['violet', 'indigo', 'blue', 'green', 'yellow', 'orange', 'red']
```

Функції для роботи зі списками та методи списків

```
len(rainbow)              # 7
len([1, ['Yes', 'No'], 2.1, 'Maybe']) # 4
len([])                   # 0
```

```
someList = [1, 2, 'three']
someList.pop() # вилучає і повертає елемент з кінця списку: [1, 2]
someList.append(3) # додає елемент в кінець списку: [1, 2, 3]
someList.pop(1) # вилучає і повертає елемент за індексом: [1, 3]
someList.insert(1, 2) # вставляє 2 на позицію 1: [1, 2, 3]
someList.extend([2, 4]) # [1, 2, 3, 2, 4]
someList.count(2) # повертає кількість входжень елемента: 2
someList.remove(2) # вилучає перше входження: [1, 3, 2, 4]
someList.sort() # [1, 2, 3, 4]
someList.reverse() # [4, 3, 2, 1]
someList.index(3) # повертає індекс елемента: 1
someList.clear() # очищує список: []
```

Раніше (під час вивчення рядків) розглядалися методи `split()` та `join()`. Оскільки вони також безпосередньо стосуються списків, наведемо приклади їх використання:

```
string = '-'.join(['a', 'b', 'c'])    # 'a-b-c'
lst = string.split('-')               # ['a', 'b', 'c']
```

Перевірка на входження та поелементний обхід списку

Часто (в умовних конструкціях) потрібно мати відповідь на питання, чи містить список деякий елемент. Для цього використовується оператор `in`:

```
traffic_light = ['зелений', 'жовтий', 'червоний']
color = input('Введіть колір: ')
if color in traffic_light:
    print(color, 'є серед кольорів світлофору')
else:
    print(color, 'відсутній серед кольорів світлофору')
```

Поелементний обхід списку здійснюється з використанням циклу `for`:

```
cities = ['Kyiv', 'London', 'New York']
for city in cities:
    print(city)
```

Якщо потрібен індекс елемента списку, то реалізується цикл по індексах:

```
for i in range(len(cities)):
    print(f'{i+1}. {cities[i]}')
```

Також можливий варіант використання вбудованої функції `enumerate()`:

```
for i, city in enumerate(cities, 1):
    print(f'{i}. {city}')
```

Заповнення списків під час виконання програми

Розглянутий вище спосіб утворення списку, який полягає в безпосередньому заданні його елементів у квадратних дужках, називається літеральним. На практиці часто списки формуються під час виконання програми. Продемонструємо цей

спосіб на прикладі програми, яка формує список унікальних літер (не символів!) у рядку:

```
sentence = 'Київ - столиця України'
uniques = []
for symbol in sentence.upper():
    if symbol.isalpha() and symbol not in uniques:
        uniques.append(symbol)
print(sorted(uniques))
```

Генератори списків

Генератори списків призначені для автоматизації створення списків, елементи яких утворюються згідно з певними правилами. Загальний синтаксис генератора списку:

```
[<вираз> for <змінна> in <колекція> if <умова>]
```

Наприклад, згенерувати список квадратів парних двоцифрових натуральних чисел можна так:

```
lst = [n**2 for n in range(10,100) if n%2 == 0]
```

Цей запис еквівалентний такій реалізації:

```
lst = []
for n in range(10, 100):
    if n%2 == 0:
        lst.append(n**2)
```

Інструкція `if` може бути опущена. Так, згаданий вище список можна згенерувати, замінивши перевірку остачі від ділення на 2 кроком 2:

```
lst = [n**2 for n in range(10, 100, 2)]
```

Накладення імен

Змінюваність списків (як і інших змінюваних типів) має побічний ефект, який називають накладенням імен (aliasing). Розглянемо його на прикладі Аліси і Боба, які вирішили вести облік прочитаних книг. Припустимо, що початково вони прочитали одні й ті ж книги і вирішили зберегти їх у відповідних списках:

```
alice_books = ['Кобзар', 'Енеїда']  
bob_books = aliceBooks
```

Після цього Аліса прочитала ще одну книгу, а Боб вирішив надрукувати свій список прочитаного:

```
alice_books.append('Тарас Бульба')  
print(bob_books) # ['Кобзар', 'Енеїда', 'Тарас Бульба']
```

Прочитана Алісою (але не Бобом) книга потрапила в список прочитаного Боба! Справа в тому, що Боб хотів просто скопіювати список Аліси, який на той момент співпадав з його списком прочитаного. Але в Python **для змінюваних типів операція присвоєння не створює копію об'єкта, а прив'язує нову змінну до того ж об'єкта**. Тобто, після виконання коду `bobBooks = aliceBooks` обидві змінні вказуватимуть на один і той самий об'єкт списку.

Щоб уникнути цього, потрібно зробити копію списку, використавши зріз одним із способів:

```
bob_books = alice_books[:]  
  
bob_books = list(alice_books)  
  
bob_books = alice_books.copy()
```

Для перевірки, чи змінні вказують на один і той самий об'єкт, чи на різні об'єкти, призначений оператор `is`. Логічний вираз

```
bobBooks is aliceBooks
```

поверне `True`, якщо обидві змінні вказують на один і той самий список, і `False`, якщо на різні. Оператор порівняння `==` застосовувати для цього не можна, оскільки він поверне `True` у випадку двох різних об'єктів списків, які містять однакові елементи.